JavaScript Full Notes

Simple explanations, one topic at a time: basics to DOM, async and advanced ideas

Getting Started

What is JavaScript?

JavaScript is the language that makes websites move and react. Buttons, menus and forms all use it. It also runs on servers with Node.js.

```
console.log("Hello!");
```

Where does it run?

Every browser has a JavaScript engine built in. Open the Console tab in developer tools and type code there.

```
alert("Hi");  // popup
```

Add it to a page

Put code inside a script tag, or link a .js file. Put the tag at the end of body so the page loads first.

```
<script src="app.js"></script>
```

Comments

Comments are notes for humans. The computer ignores them.

```
// one line
/* many
   lines */
```

Variables & Data Types

Variables

A variable is a labeled box that holds a value. Use let if the value will change and const if it never will. Avoid var.

```
let age = 20;
const pi = 3.14;
```

Data types

Every value has a type: string (text), number, boolean (true/false), undefined, null, symbol, bigint, and object.

```
"hi"; 42; true; null;
typeof 42;  // "number"
```

null vs undefined

undefined means no value was given yet. null means you chose to leave it empty on purpose.

```
let a;        // undefined
let b = null; // empty
```

Naming rules

Names can use letters, numbers, _ and $. They cannot start with a number. Use camelCase.

```
let userName = "Ana";
```

Operators & Type Conversion

Math operators

Use + - * / for normal math, % for the remainder, and ** for powers.

```
10 % 3;  // 1
2 ** 3;  // 8
```

Compare values

=== checks value and type together, so it is safer than ==. Always prefer ===.

```
5 === "5";  // false
5 == "5";   // true
```

Logical operators

&& means AND, || means OR, ! means NOT. Use them to combine conditions.

```
age > 18 && hasId;
!isBanned;
```

Type conversion

JavaScript sometimes changes types by itself (coercion). Convert on purpose to avoid surprises.

```
Number("10");  // 10
String(5);      // "5"
"5" + 1;        // "51"
```

Control Flow

if / else

Run code only when a condition is true. else runs when it is false. else if adds more choices.

```
if (score >= 90) log("A");
else if (score >= 60) log("B");
else log("C");
```

switch

Use switch to compare one value against many options. break stops it from running into the next case.

```
switch (day) {
  case "Mon": log("Start"); break;
  default: log("Other");
}
```

Ternary operator

A short if/else on one line: condition ? valueIfTrue : valueIfFalse.

```
const msg = ok ? "Yes" : "No";
```

Truthy & falsy

Some values count as false in conditions: false, 0, empty text, null, undefined, NaN. Everything else counts as true.

```
if ("") {}  // does not run
if ("a") {} // runs
```

Loops

for loop

Repeats code a set number of times. It has a start, a stop condition and a step.

```
for (let i = 0; i < 3; i++) {
  log(i);  // 0 1 2
}
```

while / do…while

while repeats as long as the condition is true. do…while always runs at least once.

```
let n = 0;
while (n < 3) n++;
```

for…of and for…in

for…of gives each item of an array. for…in gives each key of an object.

```
for (const x of [5,6]) log(x);
for (const k in {a:1}) log(k);
```

break & continue

break leaves the loop right away. continue skips this round and goes to the next.

```
if (i === 2) continue;
if (i === 4) break;
```

Functions

What is a function?

A function is a block of code with a name that you can run again and again. It can take inputs (parameters) and give back a result.

```
function add(a, b) {
  return a + b;
}
add(2, 3);  // 5
```

Arrow functions

A shorter way to write functions. With one expression the result returns automatically.

```
const square = x => x * x;
```

Default & rest parameters

Defaults fill in missing values. Rest (...) gathers extra inputs into an array.

```
function hi(name = "you") {}
function sum(...n) { return n.length; }
```

return & scope

return sends a value back and stops the function. Variables made inside a function stay inside it.

```
function f() {
  let secret = 1;
}
// secret is not available here
```

Functions in Depth

Callbacks

A callback is a function you give to another function to run later.

```
setTimeout(() => log("later"), 1000);
```

Higher-order functions

These are functions that take or return other functions. map, filter and forEach are examples.

```
[1,2].forEach(n => log(n));
```

Closures

An inner function remembers the variables around it, even after the outer function has finished. Useful for private data.

```
function counter() {
  let c = 0;
  return () => ++c;
}
const next = counter();
next(); next();  // 1, 2
```

IIFE & recursion

An IIFE runs as soon as it is defined. Recursion is a function calling itself, and it needs a stopping point.

```
(() => log("go"))();
const f = n => n <= 1 ? 1 : n * f(n-1);
```

Arrays

Create & read

An array is an ordered list. Counting starts at 0. length tells you how many items it holds.

```
const a = [10, 20, 30];
a[0];      // 10
a.length;  // 3
```

Add & remove

push and pop work at the end. unshift and shift work at the start. splice works anywhere.

```
a.push(40);
a.pop();
a.splice(1, 1);
```

map, filter, reduce

map changes every item. filter keeps some items. reduce turns the list into one value.

```
[1,2,3].map(x => x * 2);
[1,2,3].filter(x => x > 1);
[1,2,3].reduce((s, x) => s + x, 0);
```

Find & check

find gets the first match. includes checks if a value exists. some asks if any match. every asks if all match.

```
a.find(x => x > 10);
a.includes(20);
a.some(x => x > 25);
```

Sort & copy

sort changes the original array. Give it a compare function for numbers. slice and spread make copies.

```
[3,1,2].sort((a, b) => a - b);
const copy = [...a];
```

Strings, Numbers, Math & Date

Strings

Strings are text. You cannot change them in place, methods return new strings.

```
"Hello".toUpperCase();
" hi ".trim();
"a,b".split(",");
```

Search & slice text

Check or cut parts of text with these.

```
"JavaScript".includes("Script");
"abc".slice(1, 3);   // "bc"
"abc".replace("a", "z");
```

Numbers

JavaScript has one number type for whole numbers and decimals. toFixed rounds for display.

```
(3.14159).toFixed(2);  // "3.14"
parseInt("42px");     // 42
```

Math

The Math object helps with rounding and random numbers.

```
Math.floor(4.9);   // 4
Math.max(1, 5, 3);
Math.random();     // 0 to 1
```

Date

Date holds a moment in time. Months start at 0, so January is 0.

```
const d = new Date();
d.getFullYear();
Date.now();
```

Objects & JSON

Objects

An object stores related data as key and value pairs. A function inside an object is called a method.

```
const user = {
  name: "Ana",
  hi() { return "Hi"; }
};
user.name;
```

Change objects

You can add, update or delete keys at any time, even if the object is const.

```
user.age = 20;
delete user.age;
"name" in user;
```

Object helpers

Get all keys, values or pairs, or copy with spread.

```
Object.keys(user);
Object.entries(user);
const c = {...user};
```

Optional chaining

?. stops safely if something is missing instead of crashing.

```
user.address?.city ?? "none";
```

JSON

JSON is text that stores data. stringify turns an object into text. parse turns text back into an object.

```
const s = JSON.stringify(user);
JSON.parse(s);
```

Modern JavaScript (ES6+)

Template literals

Use backticks to put variables inside text easily.

```
`Hi ${name}, 2+2=${2+2}`
```

Destructuring

Pull values out of arrays or objects into variables in one line.

```
const [x, y] = [1, 2];
const {name, age} = user;
```

Spread & rest

Three dots can spread items out, or gather them together.

```
const all = [...a, ...b];
const o = {...user, age: 30};
```

Map & Set

A Map stores key/value pairs with any key type. A Set stores only unique values.

```
new Set([1, 1, 2]);  // 1, 2
new Map([["a", 1]]);
```

Modules

Modules split code into files. export shares code and import uses it.

```
export const pi = 3.14;
import { pi } from "./math.js";
```

DOM

What is the DOM?

The DOM is the page turned into objects that JavaScript can read and change.

```
document.title;
```

Select elements

querySelector gets the first match, querySelectorAll gets all matches. Use CSS selectors.

```
const el = document.querySelector(".box");
document.querySelectorAll("li");
```

Change the page

Change text, style and classes with these. textContent is safer than innerHTML.

```
el.textContent = "Hi";
el.style.color = "red";
el.classList.toggle("on");
```

Create & remove

Make a new element, fill it, then add it to the page.

```
const li = document.createElement("li");
li.textContent = "New";
list.append(li);
li.remove();
```

Events & Storage

Events

An event is something that happens, like a click or a key press. addEventListener runs your function when it happens.

```
btn.addEventListener("click", e => {
  log(e.target);
});
```

Event delegation

Put one listener on the parent instead of many on children. It also works for items added later.

```
list.addEventListener("click", e => {
  if (e.target.matches("li")) log("item");
});
```

Forms

Stop the page from reloading on submit with preventDefault, then read the input values.

```
form.addEventListener("submit", e => {
  e.preventDefault();
  log(input.value);
});
```

localStorage

Saves small data in the browser, even after closing it. It only stores text.

```
localStorage.setItem("k", "v");
localStorage.getItem("k");
```

Async JavaScript

Why async?

Some tasks take time, like loading data. Async lets the page keep working while it waits.

```
setTimeout(() => log("2s"), 2000);
```

Promises

A promise is a future result. It is pending, then fulfilled (success) or rejected (error). Use then and catch.

```
fetch(url)
  .then(r => r.json())
  .catch(err => log(err));
```

async / await

The easy way to use promises. await waits for the result inside an async function. Wrap in try/catch for errors.

```
async function load() {
  try {
    const r = await fetch(url);
    return await r.json();
  } catch (e) { log(e); }
}
```

Event loop

JavaScript does one thing at a time. The event loop runs finished async tasks when the stack is empty. Promises go before timers.

```
log(1);
setTimeout(() => log(2));
Promise.resolve().then(() => log(3));
// 1, 3, 2
```

APIs & Errors

What is an API?

An API is a way for programs to talk. Your page asks a server for data and gets it back, usually as JSON.

```
GET /api/users
```

Fetch GET & POST

GET reads data. POST sends data. Check response.ok to see if it worked.

```
await fetch("/api/users", {
  method: "POST",
  headers: {"Content-Type": "application/json"},
  body: JSON.stringify({name: "Ana"})
});
```

Several requests

Promise.all waits for every promise. Promise.allSettled gives all results, good or bad.

```
const [a, b] = await Promise.all([
  fetch(u1), fetch(u2)
]);
```

try / catch / throw

try runs risky code. catch handles the error. throw makes your own error. finally always runs.

```
try { throw new Error("Oops"); }
catch (e) { log(e.message); }
finally { log("done"); }
```

Advanced: this & OOP

this

this points to the object that called the function. Arrow functions use the this around them.

```
obj.hi();   // this = obj
```

call, apply, bind

These let you choose what this is. bind makes a new function with this fixed.

```
fn.call(obj, 1, 2);
fn.apply(obj, [1, 2]);
const b = fn.bind(obj);
```

Prototypes

Objects can borrow features from another object, their prototype. This is how inheritance works in JavaScript.

```
Dog.prototype.bark = () => {};
```

Classes

A class is a blueprint for objects. extends lets one class build on another. # makes a field private.

```
class Animal {
  #age = 1;
  constructor(n) { this.name = n; }
}
class Dog extends Animal {}
```

Advanced: More Ideas

Scope & hoisting

let and const stay inside their block. var and function declarations are lifted to the top, but let/const cannot be used before they are written.

```
{ let x = 1; }
// x is not defined here
```

Copy vs reference

Simple values are copied. Objects share one place in memory. structuredClone makes a full copy.

```
const a = {x: 1}, b = a;
b.x = 2;  // a.x is 2
structuredClone(a);
```

Regular expressions

A pattern for finding text. g means all matches, i ignores case.

```
"a1b2".match(/\d/g);  // ["1","2"]
```

Debounce

Wait until the user stops typing or clicking before running a function.

```
const debounce = (fn, ms) => {
  let t;
  return (...a) => {
    clearTimeout(t);
    t = setTimeout(() => fn(...a), ms);
  };
};
```

Iterators, Generators & Special Objects

Symbol

A Symbol is a value that is always unique. It is often used as a hidden object key.

```
const id = Symbol("id");
const o = {[id]: 1};
```

Iterators & generators

An iterator hands out values one at a time. A generator function (function*) pauses at each yield and continues later.

```
function* nums() {
  yield 1; yield 2;
}
[...nums()];  // [1, 2]
```

Getters & setters

These look like normal properties but run code when you read or write them.

```
const p = {
  first: "Ana",
  get hi() { return "Hi " + this.first; }
};
p.hi;
```

WeakMap & Proxy

WeakMap holds objects without blocking cleanup. Proxy lets you watch or change how an object is read and written.

```
const p = new Proxy({}, {
  get: (t, k) => k in t ? t[k] : "none"
});
```

Browser Web APIs

Cookies & sessionStorage

Cookies are small data sent to the server with each request. sessionStorage works like localStorage but clears when the tab closes.

```
document.cookie = "a=1";
sessionStorage.setItem("k", "v");
```

IntersectionObserver

Tells you when an element enters the screen. Good for lazy loading and scroll effects.

```
new IntersectionObserver(e => log(e))
  .observe(el);
```

Web Workers

A worker runs heavy code in the background so the page does not freeze.

```
const w = new Worker("w.js");
w.postMessage(5);
w.onmessage = e => log(e.data);
```

Location, History & Geolocation

location reads or changes the page address. history moves back and forward. Geolocation asks the user for their position.

```
location.href;
history.back();
navigator.geolocation.getCurrentPosition(log);
```

Tools, Safety & Speed

Node.js & npm

Node.js runs JavaScript outside the browser. npm installs packages made by others. Node uses require or import.

```
npm init -y
npm install lodash
node app.js
```

Debugging & testing

Use console.log, breakpoints and the debugger statement to find bugs. Tests check your code works, with tools like Jest.

```
debugger;
expect(add(2, 3)).toBe(5);
```

Security basics

Never put user text into innerHTML, it can run bad code (XSS). Never store passwords or secrets in front-end code.

```
el.textContent = userInput;  // safe
```

Speed & memory

Do less work in loops, cache results, and remove event listeners you no longer need. Forgotten references cause memory leaks.

```
btn.removeEventListener("click", fn);
```

Key Takeaways

21/21

Good habits

Use let and const. Use ===. Keep functions small. Name things clearly. Handle errors. Write comments for hard parts.

Learning path

1. Basics and control flow. 2. Functions, arrays, objects. 3. DOM and events. 4. Async and APIs. 5. Build small projects like a to-do app.

Try every example in your browser console (press F12).